🧠 Coordinate Systems in MLX Python

> Part of the MLX Python Documentation. Function details are in the MLX Reference.

Why This Topic Is Important

One of the biggest difficulties when starting graphical projects with MLX Python is understanding the difference between:

- the **logical coordinates** used by the game or maze
- the **pixel coordinates** used by the window

Most bugs in tile-based games come from mixing these two systems.

Getting this right makes it much easier to draw maps, move the player, animate entities, center the maze, detect collisions, scale the window and render assets correctly.

📦 The Two Coordinate Systems

1️⃣ Grid Coordinates (Logical)

These represent positions inside the maze or game logic. **They are not pixels** — they only describe where something is in the map structure.

```
# # # # #
# P . . #
# . # . #
# . . E #
# # # # #
```

The player `P` is on tile `(1, 1)`:

```
        player_x = 1
        player_y = 1
```

2️⃣ Screen Coordinates (Pixels)

These are the real positions inside the MLX window. MLX knows nothing about tiles — it only understands pixels.

```
        mlx.mlx_put_image_to_window(mlx_ptr, win_ptr, player_img, 128, 64)
```

This draws the image with its top-left corner **128 px from the left** and **64 px from the top**.

🔄 Converting Grid → Pixels

This is the most important formula in tile-based games:

```
TILE_SIZE = 64

screen_x = grid_x * TILE_SIZE
screen_y = grid_y * TILE_SIZE
```

Example: player at `grid (3, 2)` with `TILE_SIZE = 64`:

```
screen_x = 3 * 64   # 192
screen_y = 2 * 64   # 128

mlx.mlx_put_image_to_window(mlx_ptr, win_ptr, player_img, 192, 128)
```

🎮 Choosing a Tile Size

`TILE_SIZE` defines how big each block appears on screen. With `TILE_SIZE = 32`, every wall and floor image is 32×32 and each step moves 32 pixels on screen.

Larger tiles	Smaller tiles
✅ easier to see	✅ bigger maps fit
✅ more detailed sprites	✅ more visible area
❌ fewer tiles fit on screen	❌ harder to see details

> Your XPM assets should match the tile size. MLX does not scale images for you.

🧱 Moving by Tile

Game logic should move in grid units:

```
player_x += 1   # one TILE to the right, not one pixel
```

The pixel conversion happens **only when rendering**.

❌ Common Mistake

```
player_x += 64   # mixing pixels into game logic
```

This breaks collisions, maze indexing and pathfinding, because the coordinates no longer match the map. Keep `player_x += 1` and convert later:

```
        screen_x = player_x * TILE_SIZE
```

🧩 Centering the Maze

For a 20×10 maze with `TILE_SIZE = 64`:

```
        maze_pixel_width  = 20 * 64   # 1280
        maze_pixel_height = 10 * 64   # 640
```

In a 1600×900 window, the offset that centers it is:

```
        offset_x = (WINDOW_WIDTH  - maze_pixel_width)  // 2   # 160
        offset_y = (WINDOW_HEIGHT - maze_pixel_height) // 2   # 130
```

Every tile is then drawn with the offset added:

```
        screen_x = offset_x + grid_x * TILE_SIZE
        screen_y = offset_y + grid_y * TILE_SIZE
```

> If the maze is bigger than the window, the offset becomes negative. Either choose a smaller `TILE_SIZE` (you can compute it from `mlx_get_screen_size`) or clamp the offset to `0`.

🔁 Converting Pixels → Grid

The opposite direction is needed for mouse clicks (e.g. clicking a tile):

```
        def on_mouse(button, x, y, param):
            grid_x = (x - offset_x) // TILE_SIZE
            grid_y = (y - offset_y) // TILE_SIZE
            if 0 <= grid_x < maze_width and 0 <= grid_y < maze_height:
                print(f"Clicked tile ({grid_x}, {grid_y})")
```

Always check the bounds: a click on the margin around the maze gives a tile outside the map.

⚠️ Origin (0, 0)

In MLX, `(0, 0)` is the **top-left corner**. X grows to the right, **Y grows downward** — the opposite of a math graph.

```
(0,0) ──────────► X
  │
  │
  ▼
  Y
```

This applies to the window, to `mlx_put_image_to_window` positions, and to pixels inside an image buffer.

🐍 Recommended Architecture

Game Logic	Renderer
maze structure, collisions, movement, BFS / pathfinding	images, drawing, animation, UI, scaling
works with `(grid_x, grid_y)`	works with `(screen_x, screen_y)`

🔥 Symptoms of Mixed Coordinates

- player appears between tiles
- walls overlap or have seams between textures
- path animation misaligned with the maze
- collisions don't match what you see
- player clips through walls
- maze not centered

✅ Final Recommendation

```
Game Logic → Grid Coordinates
Rendering  → Pixel Coordinates
```

Keeping them separate makes the project cleaner, easier to debug, easier to animate and easier to scale.